Esercitazione Python n. 4 -- 21 Ottobre 2025

Obiettivo dell'esercitazione è prendere confidenza con la definizione e l'uso delle funzioni in Python, e esercitarsi ulteriormente con le istruzioni **for** e **if-then-else**.

In laboratorio, per avviare la macchina virtuale, dovete selezionarla all'interno della schermata di VirtualBox (schermata visualizzata all'accensione del PC), cliccando sull'opzione **LXLE-BIAR-4.6**

Una volta avviata la macchina virtuale, PRIMA DI INIZIARE, eseguite il programma `REGISTRAstudente.py` che si trova nella cartella `Lab1`. Inserite (separatamente) *Numero di Matricola*, *Cognome* e *Nome* seguendo le istruzioni che compaiono sul terminale, e confermate i dati che avete inserito. Il programma genera il file `studente.txt` che contiene Matricola, Cognome e Nome su tre righe separate (nell'ordine indicato). Il file `studente.txt` non deve essere modificato manualmente. Verificate che i dati nel file `studente.txt` siano corretti. In caso di errore potete rieseguire il programma `REGISTRAstudente.py`.

Scrivete la soluzione di ogni esercizio nel file .py relativo, che trovate nella cartella dell'esercitazione (ad esempio, per l'esercizio 1 scrivete il vostro programma nel file Ex1.py, per l'esercizio 2, nel file Ex2.py, e così via). Non spostate i file dalla loro posizione e non create nuovi file. Per scrivere e testare il vostro codice usate l'ambiente Spyder. Ovviamente è possibile consultare il materiale didattico disponibile sulla pagina web del corso. Si ricorda che le note relative alle lezioni Python possono essere lette con l'applicazione JupyterLab.

Esercizi

N.B.: Le funzioni devono essere completate all'interno dei file che sono stati predisposti con lo stesso nome e, come visto a lezione, possono essere testate eseguendo il file stesso, con i casi di test forniti. Notate infatti che ogni file incorpora del codice python per eseguire alcuni test sulla funzione. NON modificate questo codice, ma **SCRIVETE SOLO il contenuto della funzione**. Non spostate i file dalla loro posizione e non create nuovi file. **Si noti che, come in sede di esame, per la correzione verranno usati insiemi di dati di test diversi.**

1) **Ex1(s)** Completare la funzione Python Ex1(s) (contenuta nel file Ex1.py) realizzando una funzione che, ricevendo in ingresso una stringa **s**, restituisca una nuova stringa composta dai caratteri più frequenti nella stringa nell'ordine in cui compaiono. Ad esempio se **s** vale `'caso palese'` allora la funzione deve restituire `'ase'` in quanto i tre caratteri compaiono tutti due volte nella stringa e due è la frequenza massima (si noti che nella stringa restituita ogni carattere compare una sola volta). Nel caso in cui la **s** in input sia la stringa vuota la funzione deve restituire una stringa vuota.

2) **Ex2(s1,s2)** Completare la funzione Python Ex2(s1,s2) (contenuta nel file Ex2.py) realizzando una funzione che, ricevendo in ingresso due stringhe, **s1** ed **s2**, restituisca il numero di caratteri che compaiono lo stesso numero di volte (diverso da 0) nelle due stringhe. Ad esempio se **s1** vale `'caso palese'` e **s2** vale `'casa blue'` allora la funzione deve restituire `4`, poiché i caratteri `'c'`, `'a'`, `' '` (spazio) e `'l'` compaiono lo stesso numero di volte in **s1** e **s2**. Ovviamente se una delle due stringhe in input è vuota la funzione deve restituire `0`. Si noti che la funzione **deve** restituire sempre un valore di tipo intero.

3) **Ex3(s)** Completare la funzione Python Ex3(s) (contenuta nel file Ex3.py) realizzando una funzione che, prendendo in ingresso una stringa **s** composta di sole cifre e di lunghezza massima 10, restituisce `True` se almeno un carattere di indice *i* è uguale alla cifra corrispondente. Ad esempio, se la stringa **s** vale `'999379'` allora la funzione deve restituire `True` poiché il carattere di indice 3 è la cifra `'3'`. Se invece la stringa **s** è `'234567890'` allora la funzione deve restituire `False`, perché in nessun carattere di indice *i* è uguale alla cifra corrispondente. Ovviamente, se la stringa in input è vuota, la funzione deve restituire `False`.

4) **Ex4(s,n)** Completare la funzione Python Ex4(s,n) (contenuta nel file Ex4.py) realizzando una funzione che, prendendo in ingresso una stringa **s** ed un numero intero positivo **n** (quindi maggiore di 0), restituisce la stringa che contiene, una sola volta, i caratteri di **s** il cui codice Unicode sia esattamente divisibile per **n**. Ad esempio, se **s** è `'stringa di prova'` ed **n** vale `3`, allora la funzione deve restituire `'rio'` perché solo questi caratteri hanno la proprietà desiderata. Notate che sia la `'r'` che la `'i'` compaiono una sola volta nella stringa restituita, anche se compaiono due volte in **s**. Inoltre, nella stringa restituita i caratteri devono comparire nello stesso ordine in cui compaiono in **s**.

5) **Ex5(s)** Completare la funzione Python Ex5(s) (contenuta nel file Ex5.py) realizzando una funzione che, prendendo in input una stringa **non vuota s** contenente numeri interi positivi separati dal carattere `'-'` (trattino), restituisce il massimo intero contenuto in **s**. Ad esempio, se **s** vale `'12-123-45-6-78'` la funzione deve restituire `123`. Si assuma che `'-'`

non compaia mai in prima o ultima posizione (quindi se la stringa contiene un solo numero, non contiene alcun trattino). Si noti che la funzione **deve** restituire sempre un valore di tipo intero.

6) **Ex6(s)** Completare la funzione Python Ex6(s) (contenuta nel file Ex6.py) realizzando una funzione che riceve in ingresso una stringa **s**, e restituisce la frequenza (un numero intero) del carattere alfabetico maiuscolo che appare più volte in **s**. Se **s** non contiene alcun carattere alfabetico maiuscolo, la funzione deve restituire il numero intero 0. Ad esempio, se **s** vale `'aHa^^&^HH'`, la funzione deve restituire il numero intero `3`. Se invece **s** vale `'CIAO'`, la funzione deve restituire il numero intero `1`. Si noti che la funzione **deve** restituire sempre un valore di tipo intero.

7) **Ex7(b1,b2)** Completare la funzione Python Ex7(b1,b2) (contenuta nel file Ex7.py) realizzando una funzione che prende in input due stringhe **b1** e **b2** contenenti solo caratteri `'0'` ed `'1'` (interpretati come bit) e restituisce una stringa di `'0'` ed `'1'` che corrisponde all'AND bit a bit di **b1** e **b2** letti da sinistra a destra. Si noti che l'AND bit a bit effettua l'AND solo dei bit che occupano la stessa posizione nelle stringhe **b1** e **b2**, e per ogni posizione *i* produce 1 solo se entrambi i bit in posizione *i* in **b1** e **b2** sono pari ad 1. Per i bit di **b1** che non hanno un corrispondente bit in **b2** (in questo caso **b1** è più lunga di **b2**) la funzione deve calcolare l'AND con lo `'0'`. Analogamente nel caso in cui **b2** sia più lunga di **b1**. Ad esempio, se **b1** vale `'100'` e **b2** vale `'1011'`, la funzione deve restituire la stringa `'1000'`. Se invece **b1** vale `''` (stringa vuota) e **b2** vale `'111'` la funzione deve restituire '000'. Se entrambe **b1** e **b2** sono vuote, la funzione deve restituire la stringa vuota.

8) **Ex8(s)** Completare la funzione Python che prende in ingresso una stringa **s** e restituisce `True` se la stringa rappresenta un numero intero, `False` altrimenti. Si precisa che una stringa rappresenta un intero se non è la stringa vuota, se è composta da sole cifre, oppure comincia con un segno '+' o '-' e per il resto è composta da sole cifre. Ad esempio, se **s** vale '-2', allora deve restituire `True`, mentre se **s** vale '3.2' o '3p', allora deve restituire `False`.

9) **Ex9(n1,n2,d)** Completare la funzione Python che prende in input tre numeri interi **n1**, **n2** e **d** maggiori di 0, e restituisce il valore booleano `True` se **d** è un divisore di **n1** che NON è divisore di **n2**. Ad esempio, se **n1** vale 24, **n2** vale 6 e **d** vale 4, allora deve restituire `True`, mentre se **n1** vale 24, **n2** vale 6 e **d** vale 6, allora deve restituire `False`.

10) **Ex10(s)** Diciamo che una stringa è palindroma di livello 1 se il primo e ultimo carattere sono uguali, di livello 2 se il primo è uguale all'ultimo e il secondo al penultimo, e così via. Se la parola è palindroma, si dice che sia palindroma di livello uguale alla sua lunghezza. Completare la funzione Python **Ex10(s)** che prende in input una stringa **s** e calcola il livello massimo di palindromicità della stringa. Ovviamente, se la stringa ha il primo e ultimo carattere diversi, allora la funzione restituisce 0. Ad esempio, se la stringa **s** vale 'alidfefcila', allora la funzione restituisce è 3, se **s** vale 'ailatiditalia', allora restituisce 13 e se **s** vale 'carlo', allora restituisce 0.